Notification System Workflow

System: Decentralized Notification Logic

1. The Core Logic ("The Brain")

Before defining role-specific alerts, it is essential to understand the underlying logic mechanism that triggers a notification (ping) within the PyGram ecosystem.

1. **Action Occurs:** A user interacts with the system (e.g., clicks "Publish Timetable", "Submit Assignment", or "Request Event").
2. **Target Resolution:** The system calculates the specific recipients based on context (e.g., "All students currently enrolled in Batch A", "The Head of Department (HOD) of Computer Science").
3. **Priority Check:** The system assigns a priority level:
   - **High:** Exam rescheduling, Proctoring warnings, Major locking.
   - **Low:** Library notices, General announcements.
4. **Delivery & Tracking:** The record is inserted into notifications (for the user's UI feed) AND specific tracking tables (e.g., assignment_notification_tracking) for analytics and audit trails.

2. Scenario-Based Workflows

Scenario A: System & Policy Administration

High-level configuration changes initiated by the Superadmin.

Step	Actor	Notify Who?	Message Content
1. Policy Configuration	Superadmin	All Roles	**System Alert:** "System Policy Updated: New academic workflow rules are now in effect."
2. Maintenance Mode	Superadmin	All Roles	**Urgent:** "System maintenance scheduled for 10 PM tonight."

Scenario B: NEP Curriculum & Bucket Management

Workflows related to setting up the academic structure (Buckets/Pools) before timetables are created.

Step	Actor	Notify Who?	Message Content
1. Create NEP Bucket	College Admin	Faculty (Creator)	**Task Assigned:** "New NEP Bucket '[Bucket Name]' created. Please add course subjects."
2. Update Bucket Details	College Admin	Faculty (Publisher)	**Review Request:** "Bucket '[Bucket Name]' updated with new batch details. Please review course list."
3. Bucket Finalization	Faculty (Publisher)	Faculty (Creator)	**Status Update:** "Bucket '[Bucket Name]' approved and ready for scheduling."

Scenario C: The Timetable Lifecycle

The core scheduling loop involving Creators and Publishers.

Step	Actor	Notify Who?	Message Content
1. Generation Started	Faculty (Creator)	Faculty (Creator)	**System Alert:** "Timetable generation started. We will notify you when complete."

2. Generation Complete	System	Faculty (Creator)	**Success:** "Generation Task #123 Completed. Fitness Score: 98.5%."
3. Submission for Review	Faculty (Creator)	Faculty (Publisher) & Admin	**Action Required:** "Timetable 'B.Ed Sem 1' is pending your approval."
4. Rejection (With Feedback)	Faculty (Publisher)	Faculty (Creator)	**Action Required:** "Timetable rejected. Feedback: [Publisher's Note]."
5. Publishing (Success)	Faculty (Publisher)	Students & All Faculty	**Info:** "New Timetable published for Semester 1 effective from Oct 25th."
6. Review Deadline Missed	System (Auto)	Faculty (Publisher) & Admin	**Alert:** "Review deadline missed for [Batch Name] timetable. Please expidite."

Scenario D: NEP Student Major Selection

Critical compliance flows for student subject selection.

Step	Actor	Notify Who?	Message Content
1. Major Selection Lock	Student	Student	**Confirmation:** "MAJOR Locked: You have selected [Domain]. This is

			permanent."
2. Lock Logging	System	College Admin	**Audit:** "Student [Name] has permanently locked MAJOR: [Domain]."
3. Validation Error	System	Student	**Error Alert:** "Selection Failed: You cannot switch domains after Semester 3."
4. Major Change Request	Student	College Admin	**Request:** "Student [Name] has requested a change to their MAJOR selection."
5. Change Processed	Admin	Student	**Update:** "Your MAJOR change request has been processed. Check dashboard."

Scenario E: Assignments & Proctoring

High-frequency events requiring immediate delivery.

Event	Notify Who?	Message Content
Assignment Creation	Students (Targeted Batch)	**New Assignment:** "Data Structures due on Friday."
Proctoring Violation	Student (Immediate), Faculty (Silent Log)	**Warning:** "Tab switch detected. 2 violations remaining before auto-submit."
Submission & Grading	Student	**Result:** "Assignment

		Graded. You scored 45/50."

3. Role-Based Notification Summary

👑 Super Admin

- **Receives:**
  - System-wide critical alerts (Server load, Database failures).
  - New College Registration requests.
- **Triggers:**
  - **Policy Updates:** Notifies *All Roles* when global configurations change.

🏛 College Admin

- **Receives:**
  - **Major Lock Logs:** Audits of which students locked their majors.
  - **Major Change Requests:** Alerts when students request changes to locked subjects.
  - **Review Deadlines:** Alerts when a Publisher misses a review deadline on the timetable.
  - **Approval Requests:** Timetables pending review.
  - Conflict Alerts: When two departments attempt to book the same room/resource.
  - Faculty Activity: High-level summaries of published content.
- **Triggers:**
  - **NEP Bucket Creation:** Notifies *Creator Faculty* to add subjects.
  - **Bucket Updates:** Notifies *Publisher Faculty* to review changes.
  - College-wide announcements (e.g., "Holiday declared tomorrow").
  - Approve/Reject decisions on workflows.

🎓 Faculty (Creator)

- **Receives:**
  - **Task Assignments:** "New Bucket Created - Add Subjects."
  - **Rejection Alerts:** "Timetable rejected with feedback."
  - **Generation Results:** "Algorithm finished."
  - Workflow Updates: "Your draft was approved" or "Your draft was rejected".
- **Triggers:**
  - **Submit for Review:** Notifies Publisher/Admin.
  - Assignments, Timetable Drafts, Event Requests.

🎓 Faculty (Publisher)

- **Receives:**
  - **Review Requests:** "Bucket updated" or "Timetable pending approval."
  - **Deadline Warnings:** "Review deadline missed."
- **Triggers:**
  - **Publish:** Notifies Students/All Faculty.
  - **Reject:** Notifies Creator Faculty.

🎒 Student

- **Receives:**
  - **Timetable Published:** "New schedule available."
  - Schedule Updates: "Timetable updated. Class cancelled tomorrow at 9 AM."
  - Events: "You successfully registered for the Workshop."
  - **Major Locked:** Confirmation of permanent selection.
  - **Deadlines/Grades:** Assignment alerts.
  - Results: "Your grades for Mid-Sem have been released."
- **Triggers:**
  - **Select Major:** Notifies Admin (Lock Log).
  - **Request Change:** Notifies Admin, creator.
  - Submitting work (triggers a receipt notification).

4. Debugging & Loopholes (Critical)

Developers must address these specific gaps in the algorithm to prevent logic errors.

1. **The "Notification Spam" Loophole**
   - **Issue:** The schema has assignment_notification_tracking and notifications.
   - **Fix:** Ensure the notifications table is the sole source for the UI "Bell Icon". Use tracking tables only for email/push delivery analytics. Do not alert the user twice for the same event.
2. **The Conflict Resolution Void**
   - **Issue:** In cross_department_conflicts, a conflict is detected but often ignored.
   - **Fix:** The workflow must explicitly notify **BOTH Department HODs** involved in the conflict. Do not let the record sit unseen in the database.
3. **The "Ghost" Faculty**
   - **Issue:** A timetable is published where a class is assigned to "Guest Faculty" or "TBD" (making user_id null).
   - **Fix:** Wrap notification logic in a check: if (faculty_id != null) sendNotification(). This prevents null pointer exceptions.
4. **NEP Major Locking**
   - **Issue:** The check_major_lock_constraint database function raises an exception (Error) if a student tries to change a locked Major.
   - **Fix:** An error is not a notification. Implement a **Proactive Notification** workflow:

Send a warning notification 1 week *before* Semester 3 starts: "Your Major selection will lock permanently on [Date]. Review now."

5. **Assignment Updates**
    - **Issue:** A faculty member edits an assignment (e.g., changes the due date).
    - **Fix:** The application logic must detect the difference (old.due_date != new.due_date) and trigger a specific "Schedule Update" notification to students.
6. **The Redundant Tables**
    - **Issue:** assignment_notifications table defines the alert settings, while notifications stores the actual user alerts.
    - **Fix:** Treat assignment_notifications as the *configuration* (template) and notifications as the *delivery* (instance). Never display alerts from both tables simultaneously in the frontend.